

11 - VQC Konvergenz-Analyse: Idealer Simulator vs. Noise Models

Dieses Notebook untersucht das Trainingsverhalten des VQC unter verschiedenen Bedingungen. Im Fokus steht nicht die finale Accuracy, sondern **wie** der VQC konvergiert:

- **Idealer Simulator:** schnelle, glatte Konvergenz
- **Noise Model (ibm_marrakesh):** realistisches Hardware-Rauschprofil - konsistent mit NB 06/07
- **Noise Model (synthetisch):** vereinfachtes Depolarisierungsmodell - didaktischer Vergleich
- **IBM Hardware** (auskommentiert): zu teuer für viele Iterationen

Datensatz: Iris (3 Klassen, 2 Features [0,2], fairer Split - analog zu NB 02/06)

Variable: Anzahl Iterationen (maxiter)

Metrik: Cost/Loss pro Iteration (Callback)

Hinweis zu den Noise Models:

Das ibm_marrakesh Noise Model basiert auf echten Hardware-Kalibrierungsdaten (Gate-Fehler, Dekohärenz) und ist konsistent mit NB 06/07. Das synthetische Modell verwendet ein vereinfachtes Depolarisierungsmodell (1Q: 0.1%, 2Q: 1%) als didaktischen Vergleich.

Imports & Setup

```
In [17]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit_machine_learning.optimizers import SPSSA
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit_aer.primitives import SamplerV2
from qiskit.primitives import StatevectorSampler
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit_ibm_runtime import QiskitRuntimeService
```

```
print("Imports OK")
```

Imports OK

Datenvorbereitung (einheitliche Pipeline)

```
In [18]: # Iris - Features Index [0, 2] (sepal length + petal length)
# Konsistent mit NB 02, 03, 06, 09
iris = load_iris()
X = iris.data[:, [0, 2]]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(f"Trainingssamples: {len(X_train)}, Testsamples: {len(X_test)}")
```

Trainingssamples: 105, Testsamples: 45

Callback zur Verlaufsspeicherung

```
In [19]: def make_callback(cost_history):
def callback(nfev, x, fx, dx, accept):
    cost_history.append(fx)
return callback
```

Parameter

```
In [20]: MAXITER    = 200    # Iterationen fuer den Vergleich
SHOTS      = 1024
NUM_QUBITS = 2

# Neue API (nicht deprecated)
feature_map = zz_feature_map(feature_dimension=NUM_QUBITS, reps=2)
ansatz      = real_amplitudes(num_qubits=NUM_QUBITS, reps=2)

print(f"Feature Map: zz_feature_map, reps=2")
print(f"Ansatz: real_amplitudes, reps=2")
print(f"Trainierbare Parameter: {ansatz.num_parameters}")
```

Feature Map: zz_feature_map, reps=2

Ansatz: real_amplitudes, reps=2

Trainierbare Parameter: 6

1. Idealer Simulator

```
In [21]: cost_ideal = []

sampler_ideal = StatevectorSampler()

vqc_ideal = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=SPSA(maxiter=MAXITER),
    sampler=sampler_ideal,
    callback=make_callback(cost_ideal)
)

vqc_ideal.fit(X_train, y_train)
acc_ideal = vqc_ideal.score(X_test, y_test)
print(f"Idealer Simulator - Accuracy: {acc_ideal:.4f}, Iterationen: {len(cost_ideal)}")
```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Idealer Simulator - Accuracy: 0.4444, Iterationen: 200

2. Noise Model Simulator - ibm_marrakesh (realistisch)

Konsistent mit NB 06 und NB 07: Noise Model direkt von IBM-Hardware geladen.

Kein QPU-Kontingent wird verbraucht - laeuft lokal auf AerSimulator.

```
In [22]: cost_noise_hw = []

# IBM Account verbinden und Noise Model laden
service = QiskitRuntimeService()
hw_backend = service.backend("ibm_kingston")
noise_model_hw = NoiseModel.from_backend(hw_backend)

print(f"Noise Model geladen von: {hw_backend.name}")
print(f"Basis-Gates: {noise_model_hw.basis_gates}")

# AerSimulator mit echtem Noise Model
noisy_backend_hw = AerSimulator(noise_model=noise_model_hw)
pm_hw = generate_preset_pass_manager(optimization_level=1, backend=noisy_backend_hw)
sampler_noise_hw = SamplerV2.from_backend(noisy_backend_hw)

vqc_noise_hw = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=SPSA(maxiter=MAXITER),
    sampler=sampler_noise_hw,
    pass_manager=pm_hw,
    callback=make_callback(cost_noise_hw)
)

vqc_noise_hw.fit(X_train, y_train)
acc_noise_hw = vqc_noise_hw.score(X_test, y_test)
print(f"Noise Model (ibm_marrakesh) - Accuracy: {acc_noise_hw:.4f}, Iterationen: {len(cost_noise_hw)}")
```

```
qiskit_runtime_service.__init__:WARNING:2026-05-04 11:51:55,840: Instance was not set at service instantiation. Free and trial plan instances will be prioritized. Based on the following filters: (tags: None, region: us-east, eu-de), and available plans: (open), the available account instances are: open-instance. If you need a specific instance set it explicitly either by using a saved account with a saved default instance or passing it in directly to QiskitRuntimeService().
```

```
qiskit_runtime_service.backends:WARNING:2026-05-04 11:51:55,841: Using instance: open-instance, plan: open
```

```
Noise Model geladen von: ibm_kingston
```

```
Basis-Gates: ['cz', 'delay', 'id', 'if_else', 'measure', 'measure_2', 'reset', 'rz', 'sx', 'x']
```

KeyboardInterrupt

Traceback (most recent call last)

Cell In[22], line 25

```
21     pass_manager=pm_hw,
22     callback=make_callback(cost_noise_hw)
23 )
24
--> 25 vqc_noise_hw.fit(X_train, y_train)
26 acc_noise_hw = vqc_noise_hw.score(X_test, y_test)
27 print(f"Noise Model (ibm_marrakesh) - Accuracy: {acc_noise_hw:.4f},
Iterationen: {len(cost_noise_hw)}")
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/algorithms/trainable_model.py:202, in TrainableModel.fit(self, X, y)

```
199 if not self._warm_start:
200     self._fit_result = None
--> 202 self._fit_result = self._fit_internal(X, y)
203 return self
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/algorithms/classifiers/vqc.py:194, in VQC._fit_internal(self, X, y)

```
191     self._neural_network.set_interpret(self.interpret, num_classes)
193 function = self._create_objective(X, y)
--> 194 return self._minimize(function)
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/algorithms/trainable_model.py:299, in TrainableModel._minimize(self, function)

```
295     optimizer_result = self._optimizer( # type: ignore[call-arg]
296         fun=objective, x0=initial_point, jac=function.gradient
297     )
298 else:
--> 299     optimizer_result = self._optimizer.minimize(
300         fun=objective,
301         x0=initial_point,
302         jac=function.gradient, # type: ignore[arg-type]
303     )
304 return optimizer_result
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/optimizers/spsa.py:607, in SPSSA.minimize(self, fun, x0, jac, bounds)

```
605     if not self.blocking:
606         self._nfev += 1
--> 607         fx_next = fun(x_next)
609     self.callback(
610         self._nfev, # number of function evals
611         x_next, # next parameters
612         True,
613     ) # accepted
614 # update parameters
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/algorithms/trainable_model.py:274, in TrainableModel._get_objective.<locals>.objective(objective_weights)

```
273 def objective(objective_weights):
--> 274     objective_value = function.objective(objective_weights)
```

```
275     if isinstance(self._optimizer, SciPyOptimizer):
276         self._callback(objective_weights, objective_value)
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/algorithms/objective_functions.py:190, in OneHotObjectiveFunction.objective(self, weights)

```
188 def objective(self, weights: np.ndarray) -> float:
189     # probabilities is of shape (N, num_outputs)
--> 190     probs = self._neural_network_forward(weights)
191     # float(...) is for mypy compliance
192     value = float(np.sum(self._loss(probs, self._y)) / self._num_samples)
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/algorithms/objective_functions.py:101, in ObjectiveFunction._neural_network_forward(self, weights)

```
96 # if we get the same weights, we don't compute the forward pass again.
97 if self._last_forward_weights is None or (
98     not np.all(np.isclose(weights, self._last_forward_weights))
99 ):
100     # compute forward and cache the results for re-use in backward
--> 101     self._last_forward = self._neural_network.forward(self._X, weights)
102     # a copy avoids keeping a reference to the same array, so we are sure we have
103     # different arrays on the next iteration.
104     self._last_forward_weights = np.copy(weights)
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/neural_networks/neural_network.py:229, in NeuralNetwork.forward(self, input_data, weights)

```
227 input_, shape = self._validate_input(input_data)
228 weights_ = self._validate_weights(weights)
--> 229 output_data = self._forward(input_, weights_)
230 return self._validate_forward_output(output_data, shape)
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_machine_learning/neural_networks/sampler_qnn.py:483, in SamplerQNN._forward(self, input_data, weights)

```
481 job = self.sampler.run([(self._circuit, parameter_values[:num_samples])])
482 try:
--> 483     results = job.result()
484 except Exception as exc:
485     raise QiskitMachineLearningError(f"Sampler job failed: {exc}") from exc
```

File ~/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit/primitives/primitive_job.py:70, in PrimitiveJob.result(self)

```
68 if self._result is None:
69     self._check_submitted()
--> 70     self._result = self._future.result()
71 return self._result
```

File ~/anaconda3/envs/qml/lib/python3.11/concurrent/futures/_base.py:451, in

```

Future.result(self, timeout)
    448 elif self._state == FINISHED:
    449     return self.__get_result()
--> 451 self._condition.wait(timeout)
    453 if self._state in [CANCELLED, CANCELLED_AND_NOTIFIED]:
    454     raise CanceledError()

File ~/anaconda3/envs/qml/lib/python3.11/threading.py:327, in Condition.wait
(self, timeout)
    325 try: # restore state no matter what (e.g., KeyboardInterrupt)
    326     if timeout is None:
--> 327         waiter.acquire()
    328         gotit = True
    329     else:

KeyboardInterrupt:

```

3. Noise Model Simulator - Synthetisch (didaktischer Vergleich)

Vereinfachtes Depolarisierungsmodell (1Q-Fehler: 0.1%, 2Q-Fehler: 1%).

Zeigt den Effekt von kontrollierbarem, hardware-unabh ngigem Rauschen.

```

In [ ]: cost_noise_syn = []

# Synthetisches Noise Model
noise_model_syn = NoiseModel()
error_1q = depolarizing_error(0.001, 1)
error_2q = depolarizing_error(0.01, 2)
noise_model_syn.add_all_qubit_quantum_error(error_1q, ['h', 'x', 'u1', 'u2',
noise_model_syn.add_all_qubit_quantum_error(error_2q, ['cx'])

noisy_backend_syn = AerSimulator(noise_model=noise_model_syn)
pm_syn = generate_preset_pass_manager(optimization_level=1, backend=noisy_ba
sampler_noise_syn = SamplerV2.from_backend(noisy_backend_syn)

vqc_noise_syn = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=SPSA(maxiter=MAXITER),
    sampler=sampler_noise_syn,
    pass_manager=pm_syn,
    callback=make_callback(cost_noise_syn)
)

vqc_noise_syn.fit(X_train, y_train)
acc_noise_syn = vqc_noise_syn.score(X_test, y_test)
print(f"Noise Model (synthetisch) - Accuracy: {acc_noise_syn:.4f}, Iteration

```

4. IBM Hardware (auskommentiert)

Hinweis: Hardware-Jobs sind zu kostspielig fuer viele Iterationen (10 min QPU/Monat Open Plan).

Bei SPSA mit 200 Iterationen waeren ~19 Minuten QPU-Zeit noetig - uebersteigt das Kontingent.

Dieser Block ist fuer eine spaetere Untersuchung vorbereitet - im Text der Studienarbeit ansprechen.

```
In [ ]: # cost_hardware = []
#
# from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as HWSampler
# from qiskit.transpiler.preset_passmanagers import generate_preset_pass_mar
#
# service = QiskitRuntimeService()
# hw_backend = service.backend('ibm_kingston')
#
# pm = generate_preset_pass_manager(optimization_level=1, backend=hw_backend)
# sampler_hw = HWSampler(hw_backend)
#
# vqc_hw = VQC(
#     feature_map=feature_map,
#     ansatz=ansatz,
#     optimizer=SPSA(maxiter=MAXITER),
#     sampler=sampler_hw,
#     pass_manager=pm,
#     callback=make_callback(cost_hardware)
# )
#
# vqc_hw.fit(X_train, y_train)
# acc_hw = vqc_hw.score(X_test, y_test)
# print(f"IBM Hardware - Accuracy: {acc_hw:.4f}, Iterationen: {len(cost_hardware)}")
```

Visualisierung: Konvergenzverlauf

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('VQC Konvergenzverlauf: Idealer Simulator vs. Noise Models',
             fontsize=13, fontweight='bold')

# --- Einzelne Verlaeuft ---
ax1 = axes[0]
ax1.plot(cost_ideal, label=f'Ideal (Acc: {acc_ideal:.3f})',
ax1.plot(cost_noise_hw, label=f'Noise ibm_marrakesh (Acc: {acc_noise_hw:.3f})',
ax1.plot(cost_noise_syn, label=f'Noise synthetisch (Acc: {acc_noise_syn:.3f})',
ax1.set_xlabel('Iteration')
ax1.set_ylabel('Cost')
ax1.set_title('Cost-Verlauf pro Iteration')
ax1.legend()
ax1.grid(alpha=0.3)

# --- Geglaettete Verlaeuft (gleitender Durchschnitt) ---
ax2 = axes[1]
window = 10
```



```

def smooth(data, w):
    return np.convolve(data, np.ones(w)/w, mode='valid')

if len(cost_ideal) >= window:
    ax2.plot(smooth(cost_ideal, window), label=f'Ideal (geglaettet)',
if len(cost_noise_hw) >= window:
    ax2.plot(smooth(cost_noise_hw, window), label=f'Noise ibm_marrakesh (ge
if len(cost_noise_syn) >= window:
    ax2.plot(smooth(cost_noise_syn, window), label=f'Noise synthetisch (gegl

ax2.set_xlabel('Iteration')
ax2.set_ylabel('Cost (geglaettet)')
ax2.set_title(f'Geglaetteter Verlauf (Fenster={window})')
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('vqc_konvergenz.png', dpi=150, bbox_inches='tight')
plt.show()
print("Grafik gespeichert: vqc_konvergenz.png")

```

Zusammenfassung

```

In [ ]: print("=" * 60)
print(f"{'Bedingung':<35} {'Accuracy':>10} {'Iterationen':>12}")
print("-" * 60)
print(f"{'Idealer Simulator':<35} {acc_ideal:>10.4f} {len(cost_ideal):>12}")
print(f"{'Noise Model (ibm_marrakesh)':<35} {acc_noise_hw:>10.4f} {len(cost_
print(f"{'Noise Model (synthetisch)':<35} {acc_noise_syn:>10.4f} {len(cost_r
print("=" * 60)
print()
print("Hinweis: IBM Hardware wurde aufgrund des begrenzten")
print("QPU-Kontingents (10 min/Monat, Open Plan) nicht ausgefuehrt.")
print()
print("Noise Model-Typen:")
print("  ibm_marrakesh: echtes Hardware-Profil (konsistent mit NB 06/07)")
print("  synthetisch: vereinfachtes Depolarisierungsmodell (didaktisch)")

```